

Система поддержки на основе тикетов

Веб-приложение для автоматизации процесса обработки обращений клиентов с возможностью отслеживания статуса и истории коммуникации





ОБЗОР ПРОЕКТА

Архитектура решения



Single Page Application

Быстрая навигация без перезагрузки страницы, современный подход к разработке интерфейсов



Тикет-система

Централизованная обработка обращений с отслеживанием статуса каждого запроса



Ролевая модель

Разграничение прав доступа для клиентов, агентов и администраторов

Возможности системы по ролям

Клиенты

- Создание обращений
- Просмотр своих тикетов
- Фильтрация и поиск
- Отслеживание статуса
- Переписка с агентами

Агенты

- Обработка тикетов
- Изменение статусов
- Ответы клиентам
- Внутренние заметки
- Управление очередью

Администраторы

- Полный контроль тикетов
- Управление агентами
- Настройка категорий
- Переназначение задач
- Эскалация обращений



Современный технологический стек



React 19.2.0

Современная библиотека для создания интерактивных пользовательских интерфейсов с компонентным подходом



TypeScript 5.9.3

Строгая типизация для повышения надежности кода и улучшения опыта разработки



Vite 7.3.0

Быстрый инструмент сборки нового поколения с мгновенным горячим обновлением модулей



MobX 6.15.0

Простое и масштабируемое управление состоянием приложения с реактивным программированием



Firebase 12.7.0

Realtime Database для синхронизации данных в реальном времени между клиентами



SCSS/SASS 1.97.1

Мощный CSS-препроцессор для создания модульных и переиспользуемых стилей

Архитектура проекта

Модульная структура с четким разделением ответственности компонентов

1

Components

Переиспользуемые UI компоненты: Button, Input, Modal, Table, Layout элементы для построения интерфейса

2

Pages

Страницы приложения: HomePage, TicketsPage, AgentsPage, CustomersPage, AdminPage с логикой представления

3

Store

MobX хранилища: AuthStore, DataStore, NavigationStore, UIStore для управления состоянием

4

Types

TypeScript интерфейсы и типы для всех сущностей системы: пользователи, тикеты, сообщения

5

Styles

Глобальные стили, переменные, миксины и корпоративная сине-зелёная цветовая палитра

MobX Stores: четыре ключевых хранилища



AuthStore

Авторизация пользователей, управление ролями и правами доступа в системе



DataStore

CRUD операции с данными, взаимодействие с Firebase Realtime Database



NavigationStore

Управление навигацией между страницами без использования react-router



UIStore

Состояние пользовательского интерфейса: модальные окна, уведомления

Структура данных системы

Ticket (Тикет)

- id, subject, description
- categoryId, customerId
- agentId, status
- priority, timestamps

Message (Сообщение)

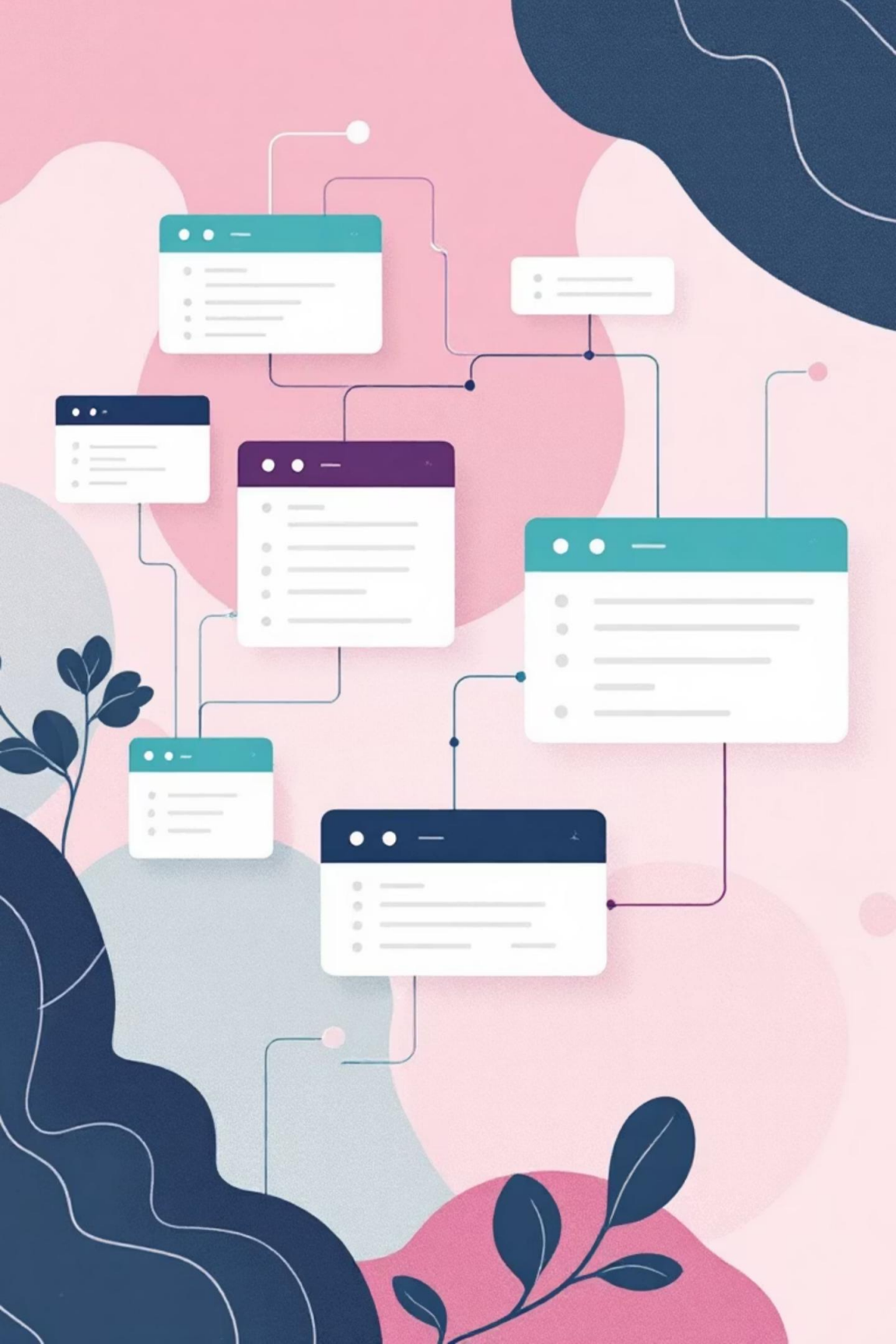
- id, ticketId, authorId
- content, isInternal
- createdAt, isActive

Agent (Агент)

- id, name, email
- department, status
- ticketsCount, isActive

Category (Категория)

- id, name, description
- slaHours, isActive





 БЕЗОПАСНОСТЬ

Система ролей и прав доступа



Customer (Клиент)

Роль по умолчанию — создание тикетов, просмотр своих обращений, добавление сообщений



Agent (Агент)

Требует авторизацию — все права клиента + обработка тикетов, изменение статусов, ответы клиентам



Admin (Администратор)

Полный доступ — все права агента + управление агентами, категориями, полный контроль данных

Ключевые преимущества решения

Масштабируемость

Модульная архитектура позволяет легко расширять функциональность и добавлять новые возможности

Производительность

Vite и React обеспечивают быструю загрузку и отзывчивый интерфейс даже при большом объеме данных

Типобезопасность

TypeScript исключает множество ошибок на этапе разработки и улучшает поддерживаемость кода

Realtime обновления

Firebase синхронизирует данные в реальном времени между всеми клиентами системы



Итоги и результаты

Разработанная система представляет собой **полнофункциональное решение** для автоматизации технической поддержки с современным стеком технологий.

1 Реализована ролевая модель

с разграничением прав доступа

2 Создана модульная архитектура

с четким разделением ответственности

3 Применены современные технологии

для высокой производительности

Запуск проекта: `npm install` → `npm run dev`

